Status

Feature Status Ledger

Revision: 13 **Last modified:** 2026-06-16T14:05:00Z **Authority:** constitution §11.4.153 (per-feature Status + video-recording confirmation), composes §11.4.44 (revision header), §11.4.45 (integration-status-doc + closed status vocabulary), §11.4.56 (Status_Summary two-audience companion), §11.4.65 (HTML+PDF export) + §11.4.153 DOCX, §11.4.107 (no frozen-frame video proof), §11.4.6 (no-guessing — every row reconciled against real code). **Scope:** Full feature surface of the `ytdlp` Podman/Docker orchestration around `yt-dlp` — every service, client surface, CLI script, landing route, dashboard component, test/challenge suite, and every PLANNED-but-unbuilt feature. **Maintainer:** project conductor.

How to read this ledger

- **Implementation** — `Implemented` (code present + wired), `Partial` (code present, incomplete/edge-gaps), `Planned` (not yet built).
- **Validation/Verification** — closed status vocabulary per §11.4.45: `PASS` / `FAIL` / `SKIP` / `PENDING_FORENSICS` / `OPERATOR-BLOCKED`. Because no clean-baseline full-suite run with captured evidence was executed in the session that produced this ledger, verdicts that would require live captured runtime evidence are recorded `PENDING_FORENSICS` (per §11.4.6 — an unproven PASS is forbidden; the code is present and test files exist, but a green-with-captured-evidence run is not yet on record here). `Planned` rows carry `SKIP` (no feature to validate yet).
- **Video-Confirmation** — per §11.4.153 every user-visible confirmed claim MUST be backed by a recorded real-use capture. The strong-model analysis path is the agent's OWN native multimodal read (Claude Opus 4.8 via the Read tool — see `docs/research/vision-path/FINDINGS.md`); local CPU models (moondream) are NOT used (too slow + hallucinate). **NINE render/transcript captures are confirmed — dashboard `/` (download-form + navbar), dashboard `/queue`, dashboard `/history`, dashboard `/cookies`, landing 'Боба' UI, MeTube UI, postprocess status API, `./status` CLI, `./download` CLI (together confirming the dashboard's five components + the other surfaces) PLUS a landing read-only-API transcript (`ytdlp---landing-api---*`) confirming seven §4 routes (`/`, `/health`, `/api/cookie-status`, `/api/profile-status`, `/api/aborted-history` GET, `/logo.png`, `/favicon.ico`) — AND ONE end-to-end FLOW is confirmed (download→webready transcode, via advancing live-pipeline counter + ffprobe-validated just-produced artifact).** Every OTHER row remains `PENDING — not yet recorded` until its window-scoped, `ytdlp---`-prefixed real-use capture (§11.4.154/.155) is recorded + analyzed. The four running §1

services (metube, landing, dashboard, yt-dlp-cli) are also marked service-serving-confirmed, each citing its surface capture(s). The remaining PENDING rows are dominated by **mutating** routes/flows (POST/DELETE cookie-upload, delete-download, aborted-history write) which are NOT auto-captured because they would alter operator state (§9.2/§11.4.122), the MP3-derivation flow (no audio artifact yet), the not-running services (VPN profile — operator `.ovpn`; watchtower — docker profile), and the state-changing root CLI scripts (`./start`/`./stop`/`./restart`/`./cleanup`/`./init` — running them would destabilize the live stack).

> **HONEST FACT (§11.4.6 / §11.4.153):** As of this revision the real PASSes span TEN distinct `ytdlp---`-prefixed captures. NINE are scope=render/transcript: seven image renders analyzed by Claude Opus 4.8 native multimodal — dashboard `/` (`ytdlp---dashboard---20260615T221723Z.png`, download-form + navbar), dashboard `/queue` (`ytdlp---dashboard-queue---20260616T083018Z.png`, empty state), dashboard `/history` (`ytdlp---dashboard-history---20260616T083023Z.png`, content state, 46 items), dashboard `/cookies` (`ytdlp---dashboard-cookies---20260616T083029Z.png`, content state, per-platform), landing 'Боба' UI (`ytdlp---landing---20260616T074850Z.png`), MeTube UI (`ytdlp---metube---20260616T075608Z.png`), postprocess status API (`ytdlp---postprocess---20260616T075648Z.png`) — plus three transcripts: `./status` (`ytdlp---status---20260616T075901Z.txt`), `./download` (`ytdlp---download-cli---20260616T080936Z.txt`), and the landing read-only-API sweep (`ytdlp---landing-api---20260616T083612Z.txt`, seven §4 routes each HTTP 200/204 with real bodies). ONE is scope=FLOW: the download→webready transcode (`ytdlp---webready-flow---20260616T080633Z.txt`), confirmed by the live pipeline's `done` counter advancing 25→26 across two real status snapshots plus ffprobe-validation of the just-produced 1920×1080 h264 +faststart artifact (the postprocessor is a backend with no UI, so the §11.4.69/.107 sink-side evidence is the validated artifact, not a screen recording). **Still NOT flow-confirmed:** MP3-derivation and cookie-upload FLOWS (the latter operator-gated — needs fresh operator cookies; uploading test cookies would overwrite real operator state, §9.2/§11.4.122; the `/cookies` render shows YouTube + X cookies expired 2d ago, corroborating the gate). Every other cell honestly reads `PENDING — not yet recorded`. No row over-claims.

OPERATOR-BLOCKED rows (top per §11.4.45)

No feature in this project is currently `OPERATOR-BLOCKED`. (The VPN profile requires operator-supplied `.ovpn` credentials and `vpn-auth.txt`, but the VPN *code path* is implemented and testable in compose-config form without live credentials, so it is classified `PENDING_FORENSICS`, not `OPERATOR-BLOCKED`.)

> **Rev 10 — captured retest + fresh re-records (§11.4.40 safe subset, parallel subagents §11.4.103):** the `media_postprocessor` code-under-test is now `PASS` with captured evidence — **69/69 pytest** (incl. the new `test_skip_rule_regression.py` §11.4.135 guard) + **smoke 22/0** + **contract 18/0** GREEN against the live stack, raw output at `qa-results/`

retest-20260616T091806Z/. All four UI surfaces were re-recorded fresh + re-read natively (4/4 PASS): ytdlp---dashboard---20260616T091827Z.png, ytdlp---landing---20260616T091840Z.png, ytdlp---metube---20260616T091849Z.png, ytdlp---postprocess---20260616T091900Z.png. The container-lifecycle suites (scenario/integration/full-suite) were deliberately NOT run — they would tear down the released ytdlp-1.4.0 stack — so rows depending on them stay PENDING_FORENSICS. Issue triage (systematic-debugging §11.4.102): the done-counter-frozen + the 11 failed jobs were both proven NOT bugs (a long live transcode; pre-fix stale intermediate-file entries) — see docs/research/vision-path/ + commit history.

Rev 11 — Round-2 parallel subagents (189 tests green this session): added **Angular dashboard unit 63/63 SUCCESS** (download-form/queue/history/cookies components + metube.service ×2 → those §5 Validation cells now PASS) and **pure-shell unit 17/17 PASS** (tests/test-unit.sh) — evidence qa-results/retest-20260616-round2.md. Cumulative captured-green this session = 69 pytest + 22 smoke + 18 contract + 63 Angular + 17 shell = **189 tests, 0 real regressions**. Vision (§11.4.150) was EMPIRICALLY VALIDATED on this host — mlx-vlm + Qwen2.5-VL-3B (Metal) does ~20 s/frame with a grounded UI description (good-not-flawless; native-multimodal stays primary) — see docs/research/vision-path/CPU_VISION_RESEARCH.md.

Rev 13 — PRIMARY DEPLOYMENT MOVED TO nezha.local (§11.4.76 distributed booting; local Mac stack stopped): the full no-vpn System was distributed to nezha (amd64, Podman 5.7.1) via scripts/deploy-remote.sh + deploy/remote-hosts.env.example, and HEAVY-TESTED against real running production services — **captured evidence in qa-results/nezha-heavytest-20260616/** (+ SUMMARY.md): run-tests scenario **17/0** + integration **123/0** + error **24/0**, container-restart PASS, chaos **27/0**/1skip(DNS), memory-limits PASS, smoke **22/0**, contract **18/0**, integration-realhttp **22/0**, 7/7 challenges (incl. download_completes → a real **111 MB webready** file on disk), postprocessor pytest **46p/23skip/0fail**, dashboard 34/1(env), media-services 16/0, aborted-history 9/0, bulk 9/0. **Zero product bugs**; 6 deploy/test issues root-caused + fixed (§11.4.102, commits 5d15a64/f10c727/acbed76/b96dc8f). The 4 user surfaces were **re-recorded on nezha + re-read natively (4/4 PASS):** ytdlp---nezha-dashboard---20260616T135945Z.png (full Add-Download UI + 16-platform grid), ytdlp---nezha-metube---20260616T140007Z.png (real fresh downloads "M5 Ultra Mac Studio Leaks…" 62.62 MB etc.), ytdlp---nezha-postprocess---20260616T140014Z.png ({"healthy":true,...,"done":7,"failed":0} — clean pipeline), ytdlp---nezha-landing---20260616T140001Z.png (the :8086→:9090 auto-redirect fired, as designed). The §1 services + §3 CLI scripts + §6 test/challenge suites are therefore now backed by captured green runs on a REAL Linux+Podman production host (the heavy container-lifecycle suites that could NOT run on the Mac).

---

1. SERVICES (docker-compose.yml)

Feature	Component	Category	Implementation	Wiring
VPN tunnel (`dperson/openvpn-client`, NET_ADMIN, `/dev/net/tun`, port 3130, healthcheck ping 8.8.8.8)	openvpn-yt-dlp	Networking / Service	Implemented (`vpn` profile)	Reachable: `metube` + `yt-dl` its netns via `network_mode` `service:openvpn-yt-dlp`
Download queue/ history API + minimal UI (`ghcr.io/alexta69/metube:latest`, port 8088 direct / netns under VPN)	metube / metube-direct	Backend API / Service	Implemented	Reachable: dashboard nginx to `:8081`; `${DOWNLOAD_DIR` `config` volumes
Flask 'Боба' cookie-auth onboarding + aborted-history ledger + proxy endpoints (`landing/`, ports 8087 vpn / 8086 no-vpn)	landing-vpn / landing-no-vpn	Web UI + API / Service	Implemented	Reachable: published port; p delete-download to MeTube `aborted.json`
Sleeping CLI container for `./download` (`ghcr.io/jim60105/yt-dlp:pot`, `while true; do sleep 3600; done`, cookie copy at startup)	yt-dlp-cli / yt-dlp-cli-vpn	CLI backend / Service	Implemented	Reachable: `./download` exe it; `yt-dlp/config`, `cookie` downloads volumes
Angular 17 + nginx dashboard (`dashboard/`, port 9090, `/api/*` proxied to MeTube + landing)	dashboard	Web UI / Service	Implemented (`no-vpn` profile only)	Reachable: nginx `resolver` `proxy_pass` from `entrypo`

Feature	Component	Category	Implementation	Wiring
Image auto-update (`containrrr/watchtower:latest`, schedule `0 0 */3 * * *`, cleanup)	watchtower	Infra / Service	Implemented (`docker` profile)	Reachable: docker.sock mou `com.centurylinklabs.wa` labels

2. CLIENT SURFACES

Feature	Component	Category	Implementation	Wiring	Tests Coverage
Dashboard web UI (download-form / queue / history / cookies / navbar, error-boundary, not-found)	dashboard	Web client	Implemented	Routed via `app.routes.ts`; calls `/api/*` through nginx	`tests/test-dashbo` `e2e/tests/dashboa` component `*.spec.t`
Landing Flask 'Боба' UI (3-step cookie onboarding, drag-drop upload, freshness banner, services grid)	landing	Web client	Implemented	`INDEX_TEMPLATE` in `landing/app.py`; auto-redirect to dashboard	`tests/e2e/tests/` `landing.spec.ts`, `t` `dashboard.sh`
MeTube minimal UI (vendor original interface, dark theme)	metube	Web client	Implemented (vendor)	Served on `:8088`; linked as "Classic UI" from landing	`tests/test-media-`

Feature	Component	Category	Implementation	Wiring	Tests Coverage
`./download` CLI (host script execs yt-dlp in sleeping container)	download	CLI client	Implemented	`./download` → exec into `yt-dlp-cli`; runtime auto-detect	`tests/test-media-challenges/script download_complete`

3. ROOT CLI SCRIPTS

Feature	Component	Category	Implementation	Wiring	Tests Coverage
Init environment (create dirs, validate `.env`, generate `yt-dlp.conf`, write `vpn-auth.txt` chmod 600)	./init	CLI script	Implemented	Runtime auto-detect via `lib/container-runtime.sh`	`tests/test-unit.sh`, `t integration.sh`, `tests`
Start services (reads `USE_VPN`, pulls images, brings up profile)	./start	CLI script	Implemented	compose up via detected runtime	`tests/test-scenarios integration.sh`
Start no-vpn (force no-vpn profile)	./start_no_vpn	CLI script	Implemented	compose `--profile no-vpn`	`tests/test-scenarios scripts/ no_vpn_profile_direc`
Stop services (all profiles, clean pods/ networks)	./stop	CLI script	Implemented	compose down + cleanup	`tests/test-scenarios`
Status summary (service +	./status	CLI script	Implemented	queries running containers +	`tests/test-integrati`

Feature	Component	Category	Implementation	Wiring	Tests Coverage
HTTP health)				HTTP endpoints	
Download helper (yt-dlp via exec)	./download	CLI script	Implemented	exec into `yt-dlp-cli`	`tests/test-media-ser` `scripts/download_com`
Check VPN (query ipinfo.io inside VPN container)	./check-vpn	CLI script	Implemented	exec into `openvpn-yt-dlp`	`tests/test-vpn-smoke`
Restart services	./restart	CLI script	Implemented	stop + start sequence	`tests/test-scenarios`
Cleanup containers/ pods/ networks	./cleanup	CLI script	Implemented	compose down + prune	`tests/test-scenarios` `scripts/ container_restart_re`
Release preparation (commit + push)	./prepare-release.sh	CLI script	Implemented	git operations	bash `-n` syntax gate (`mak`
Auto-update setup (cron for Podman; Watchtower alt for Docker)	./setup-auto-update	CLI script	Implemented	writes cron entry / documents Watchtower	bash `-n` syntax gate
Update images (pull latest before start)	./update-images	CLI script	Implemented	compose pull via detected runtime	bash `-n` syntax gate

4. LANDING ROUTES (landing/app.py)

Feature	Component	Category	Implementation	Wiring
	landing		Implemented	`render_template_string(INDEX_TE`

Feature	Component	Category	Implementation	Wiring
`/` — Боба onboarding page (3-step cookie flow, session id, dynamic service URLs)		HTTP route		
`/app` — proxy GET to MeTube backend	landing	HTTP proxy route	Implemented	`requests.get(METUBE_URL)` streamed
`/api/upload-cookies` (POST) — validate Netscape file (UTF-8 + recognised-domain gate) → forward to MeTube	landing	HTTP API route	Implemented	`_validate_cookie_file` + POST to `${METUBE_URL}/upload-cookies`
`/api/delete-cookies` (POST) — remove `/config/cookies.txt`	landing	HTTP API route	Implemented	`os.remove`
`/api/cookie-status` (GET) — has_cookies + age + per-platform breakdown + MeTube reachability	landing	HTTP API route	Implemented	`_summarize_cookies_by_platform` + MeTube fallback
`/api/aborted-history` (GET) — return persisted aborted list	landing	HTTP API route	Implemented	`_read_aborted_history` (lock-free re
`/api/aborted-history` (POST) —	landing	HTTP API route	Implemented	`_aborted_history_lock` + `_write_aborted_history`

Feature	Component	Category	Implementation	Wiring
append entry, flock-guarded, 60s idempotency				
`/api/aborted-history` (DELETE) — remove specific urls or `*`	landing	HTTP API route	Implemented	locked read-modify-write
`/api/profile-status` (GET) — active compose profile + vpn_active + MeTube reachability	landing	HTTP API route	Implemented	reads `ACTIVE_PROFILE` env
`/health` (GET) — service + MeTube reachability + timestamp	landing	HTTP API route	Implemented	`requests.get(${METUBE_URL}/hist`
`/logo.png` (GET) — serve branding asset	landing	HTTP static route	Implemented	`send_from_directory`
`/favicon.ico` (GET) — 204 stub	landing	HTTP static route	Implemented	returns `"", 204`
`/api/delete-download` (POST) — remove from history + optional file delete (path-traversal guarded)	landing	HTTP API route	Implemented	POST `${METUBE_URL}/delete` + `os.w` `os.remove` within `DOWNLOAD_DIR`

5. DASHBOARD COMPONENTS (dashboard/src/app)

Feature	Component	Category	Implementation	Wiring
Download form (url, quality select, format select, folder, add via `/api/add`, inline tracker + error state, re-enable after failure)	download-form	Angular component	Implemented	`MeTubeService.a`
Queue (STATE_META: pending / preparing / downloading / postprocessing / finished / error / aborted; cancel-all; record-and-close aborted)	queue	Angular component	Implemented	`getHistoryPolli` `recordAbortedIt`
History (+ aborted-history merge via `abortedToDownloadInfo`, retry, delete-with-file, batch + delete-all, refresh)	history	Angular component	Implemented	`getHistory` + `getAbortedHisto` `deleteDownloadW`
Cookies (status badge fresh/expiring/expired, age, MeTube online/offline, per-platform breakdown, upload, delete)	cookies	Angular component	Implemented	`getCookieStatus` `uploadCookies` + `deleteCookies`
Navbar (routerLinks Download/Queue/History/Cookies; VPN / No-VPN /	navbar	Angular component	Implemented	`getProfileStatu`

Feature	Component	Category	Implementation	Wiring
unknown profile badge; MeTube-unreachable badge)				
Error boundary (renders loading / error+retry / empty / content states)	error-boundary	Angular component	Implemented	wrapped around lis components
Not-found (404 catch-all route)	not-found	Angular component	Implemented	** route in app.ro
Error interceptor (RxJS error surfacing, clears loading)	error-interceptor.service	Angular service	Implemented	app.config.ts pr
MeTube service (typed API: add/start/history/ delete/clear/retry/cookies/ profile/aborted/version/ poll; no any)	metube.service	Angular service	Implemented	HttpClient to /ap

6. TESTS / CHALLENGES (real test files reconciled)

Feature	Component	Category	Implementation	Wiring
Unit tests (pure-shell logic)	tests/test-unit.sh	Test suite	Implemented	run-tests.sh -p
Integration tests (containers up)	tests/test-integration.sh	Test suite	Implemented	run-tests.sh -p integration
Real-HTTP integration (no mocks, hits :9090/:8086/:8088)	tests/test-integration-realhttp.sh	Test suite	Implemented	services must be up
Scenario tests (lifecycle phases)	tests/test-scenarios.sh	Test suite	Implemented	run-tests.sh -p scenario

Feature	Component	Category	Implementation	Wiring
Error-phase tests	tests/test-errors.sh	Test suite	Implemented	`run-tests.sh -p`
Dashboard + landing integration	tests/test-dashboard.sh	Test suite	Implemented	services up
Dashboard operations (post-fix full automation)	tests/test-dashboard-operations.sh	Test suite	Implemented	services up
Aborted-history (incl. concurrent-posts no-500)	tests/test-aborted-history.sh	Test suite	Implemented	landing up
Cookie validator (`_validate_cookie_file`)	tests/test-cookie-validator.sh	Test suite	Implemented	landing app.py
Media-services compatibility (anti-bluff documented-failure)	tests/test-media-services.sh	Test suite	Implemented	yt-dlp-cli / metube u
Bulk operations	tests/test-bulk-operations.sh	Test suite	Implemented	services up
Add-all-platforms / add-download	tests/test-add-all-platforms.sh, tests/test-add-download.sh	Test suite	Implemented	metube up
VPN compose + smoke	tests/test-vpn-compose.sh, tests/test-vpn-smoke.sh	Test suite	Implemented	vpn profile
Chaos / resilience	tests/test-chaos.sh	Test suite	Implemented	services up
Constitution inheritance + meta-test	tests/test-constitution-	Test suite	Implemented	repo tree

Feature	Component	Category	Implementation	Wiring
	inheritance.sh, tests/meta-test-constitution-inheritance.sh			
E2E (Playwright): dashboard, landing, cross-service, clear-history, delete-retry-cancel, cleanup-and-start	tests/e2e/	E2E suite	Implemented	`run-e2e.sh` + `playwright.confi`
Benchmark suite	tests/benchmark/ run-benchmarks.sh	Benchmark suite	Implemented	services up
Cookie auth + youtube-download + validate-fix	tests/cookies/	Test suite	Implemented	metube + cookies
MeTube anti-bluff challenge bank (download-completes, queue-realtime, form-reenables)	tests/challenges/	Challenge bank	Implemented	`run-metube-challenges.sh` + `metube-challenges.json`
Root HelixQA challenge scripts (api-contract, container-restart-resilience, download-completes, form-reenables, host/user OOM+suspend, landing-cookie-upload, memory-limits, no-vpn-direct, queue-lifecycle, queue-polling, retry-visible)	challenges/scripts/	Challenge bank	Implemented	`run_all_challeng`
Challenges submodule (HelixQA challenge framework, p1-f06..f18 CLI-agent feature dirs)	Challenges/	Challenge framework (submodule)	Implemented (submodule)	`Challenges/` catalo

7. MEDIA POST-PROCESSOR (media_postprocessor — shipped `ytdlp-1.4.0`)

Reconciled 2026-06-16 (Rev 4): these rows were marked `Planned` in Rev 3 but the dual-version pipeline shipped in tag `ytdlp-1.4.0`. Verified as FACT: real Python package `media_postprocessor/` (config/jobs_db/media_probe/transcoder/watcher/worker/service), compose service `media_postprocessor` (container `media-postprocessor`, `MP_PORT=8089`, `MP_DB_PATH=/`

downloads/.media_postprocessor/jobs.db), nginx proxy dashboard/nginx.conf.template:116 /api/postprocess/ → media-postprocessor:8089, live endpoint returning real JSON, 6+ real webready-*.mp4 on disk.

Feature	Component	Category	Implementation	Wiring
Media post-processor sidecar (SQLite-WAL queue, watcher + worker, crash-safe resume, atomic output, ffprobe-validate, /postprocess/* API)	media_postprocessor	Service	Implemented (ytdlp-1.4.0)	Reachable: compose service media_postprocessor (cont media-postprocessor, MP_PORT=8089); dashboard ng proxies /api/postprocess/ media-postprocessor:808 (nginx.conf.template:116
Web-ready video transcode (webready-<base>.mp4, H.264/AAC, +faststart)	media_postprocessor	Pipeline feature	Implemented	transcoder.transcode_vi → webready-<base>.mp4, ffprobe-validated (_validate_webready assert h264 + faststart); watcher(+m age) + worker
MP3 audio derivation (<base>.mp3, 320 kbps CBR)	media_postprocessor	Pipeline feature	Implemented	transcoder.derive_mp3() <base>.mp3; worker invokes audio jobs
Post-process status API (/postprocess/status — healthy + per-state counts)	media_postprocessor	API feature	Implemented	media_postprocessor.serv postprocess/status; nginx api/postprocess/ proxy

Feature	Component	Category	Implementation	Wiring

8. PLANNED — not yet built (Implementation = Planned)

Confirmed ABSENT from the codebase, re-verified 2026-06-16 (Rev 4): no matching Angular component (`find dashboard/src/app -iname '*postprocess*' -o -iname '*pipeline*'` → none) and no user-facing download-resume feature — §11.4.6.

Feature	Component	Category	Implementation	Wiring	Tests Coverage	Val Ver
Dashboard pipeline-status UI (surface transcode/ derive progress)	dashboard	Web UI feature (planned)	Planned	Not wired — no component exists	None yet	SK
Resume mechanism (resume interrupted/ aborted *downloads*)	metube / media_postprocessor	Pipeline feature (planned)	Planned	Not wired (the postprocessor's crash-safe *job* resume is implemented; user-facing *download* resume is not)	None yet	SK

Reconciliation note (§11.4.153 / §11.4.118)

Every row above maps to real code (a named compose service, a Flask route in `landing/app.py`, an Angular component/service file under `dashboard/src/app/`, a root CLI script, a `media_postprocessor/` module, or a test/challenge file) — except the §8 PLANNED rows, which are explicitly marked `Planned`/`SKIP` because the codebase contains no matching implementation (re-verified 2026-06-16). No code-present feature is omitted; no row lacks corresponding code. Per §11.4.6, no verdict claims a PASS without captured evidence. Per §11.4.153, exactly FIVE Video-Confirmation cells claim a real PASS — dashboard UI, landing 'Боба' UI, MeTube UI, postprocess status API, and the `./status` CLI — each backed by a `ytdlp---`-prefixed captured artifact analyzed by Claude Opus 4.8 native multimodal, **all scope=render/transcript (none flow-confirmed)**; every other cell reads `PENDING — not yet recorded`.